Studio Nx

JS 299 — 오프라인 렌더링

2026-10-20

JavaScript 299 — 오프라인 렌더링 (`audio/offline.js`)

항목	내용
파일	`studionx/frontend/static/js/audio/offline.js`, `studionx/frontend/static/js/audio/mixer.js` (`applyTracks`의 `immediate`)
작업	S1-11b WAV 출력: 재생과 같은 그래프를 `OfflineAudioContext`에 만들어 범위의 소리를 계산합니다 (FR-302, FR-306)
작성	Claude (AI), 2026-10-20, 사용자 검토
이전, 다음	이전 298 내보내기 계획 계산, 다음 300 내보내기 창

1. 하는 일

범위 안의 소리를 실시간보다 빠르게 계산해서 돌려줍니다. 재생(290)이 쓰는 `MixerGraph`, `EffectRack`, `InstrumentRack`, `ClipPlayer`, `NotePlayer`를 그대로 쓰고, 시계만 실시간 `AudioContext`에서 `OfflineAudioContext`로 바꿉니다. 그래서 들리는 소리와 파일의 소리가 같은 코드에서 나옵니다.

```
renderOffline(옵션)
  1. 길이 계산: 범위(초) + 꼬리  ─▶  new OfflineAudioContext(2, 프레임 수, 샘플레이트)
  2. 샘플 디코딩 (오프라인 컨텍스트의 샘플레이트로 맞춰짐)                    진행률 0~10%
  3. 믹서, 이펙트, 악기, 클립 재생기를 재생과 똑같이 만들고 트랙 값을 즉시 반영
  4. 2초 창마다 멈춰서(suspend) 그 창의 소리만 예약하고 다시 진행(resume)     진행률 10~100%
  5. startRendering ─▶ 채널별 Float32Array
```

2. 언제 쓰나

쓰는 곳	하는 일
`sequencer-export.js`의 `startExport` (300)	창의 옵션과 화면 스냅샷을 넘겨 렌더링을 시킵니다. 진행률과 취소 확인 함수도 함께 넘깁니다.
시험 `render_offline.mjs`	클립, 노트, 이펙트, 믹서, 범위, 꼬리, 진행률, 취소를 숫자로 확인합니다.

3. 코드

3.1 순수 함수

```
export function clipsInRange(clips, startTick, endTick) {
  const out = [];
  (clips || []).forEach(function (c) {
    const end = c.start_tick + c.length_tick;
    if (end <= startTick || c.start_tick >= endTick) return;
    const copy = {};
    Object.keys(c).forEach(function (k) { copy[k] = c[k]; });
    copy.length_tick = Math.min(end, endTick) - c.start_tick;
    out.push(copy);
  });
```

```
  return out;
}
```

```
export function windowBoundaries(startTick, endTick, baseBpm, changes, windowSec) {
  const t0 = ticksToSecondsWithChanges(startTick, baseBpm, changes);
  const total = ticksToSecondsWithChanges(endTick, baseBpm, changes) - t0;
  const out = [startTick];
  for (let k = 1; k * windowSec < total; k++) {
    const tick = Math.ceil(secondsToTicks(t0 + k * windowSec, baseBpm, changes));
    if (tick > out[out.length - 1] && tick < endTick) out.push(tick);
  }
  out.push(endTick);
  return out;
}
```

3.2 렌더링

```
export async function renderOffline(o) {
  const changes = o.tempoChanges || [];
  const progress = o.onProgress || function () {};
  const cancelled = o.isCancelled || function () { return false; };
  const Ctx = o.OfflineContext || (typeof OfflineAudioContext !== 'undefined' ? OfflineAudioContext : nu
ll);
  if (!Ctx) throw new Error('이 브라우저는 OfflineAudioContext 를 지원하지 않습니다');

  const rangeSec = ticksToSecondsWithChanges(o.endTick, o.bpm, changes) - ticksToSecondsWithChanges(o.st
artTick, o.bpm, changes);
  const totalSec = rangeSec + Math.max(0, o.tailSec || 0);
  const frames = Math.max(1, Math.ceil(totalSec * o.sampleRate));
  const ctx = new Ctx(2, frames, o.sampleRate);

  const audioClips = clipsInRange(o.audioClips, o.startTick, o.endTick);
  const midiClips = clipsInRange(o.midiClips, o.startTick, o.endTick);

  // 1) 샘플 디코딩 (오프라인 컨텍스트의 샘플레이트로 다시 맞춰짐). 전체 진행률의 앞 10%
  const buffers = {};
  const ids = [];
  audioClips.forEach(function (c) { if (c.sample_id != null && ids.indexOf(String(c.sample_id)) < 0) id
s.push(String(c.sample_id)); });
  for (let i = 0; i < ids.length; i++) {
    if (cancelled()) throw new Error('cancelled');
    try {
      const bytes = await o.loadSampleBytes(Number(ids[i]));
      buffers[ids[i]] = await ctx.decodeAudioData(bytes);
    } catch (err) {
      if (typeof console !== 'undefined') console.warn('[export] 샘플 ' + ids[i] + ' 을(를) 불러오지 못해
건너뜁니다', err);
    }
    progress(0.1 * (i + 1) / ids.length);
  }

  // 2) 재생과 같은 그래프
  const mixer = new MixerGraph(ctx);
  const effects = new EffectRack(ctx, mixer, o.getEffects, function () { return o.bpm; });
  const rack = new InstrumentRack(ctx, function (id) { return mixer.trackInput(id); }, o.getInstrument
```

```
s);
  const player = new ClipPlayer(ctx, function (id) { return mixer.trackInput(id); }, function (sid) { re
turn buffers[String(sid)] || null; });
  const notePlayer = new NotePlayer(ctx, function (id, pitch, vel, when, dur) { return rack.playNote(id,
pitch, vel, when, dur); });
  effects.sync(o.tracks);
  mixer.applyTracks(o.tracks, true);

  // 3) 창마다 소리를 예약
  const noteEvents = expandNotes(midiClips).filter(function (ev) { return ev.abs_start >= o.startTick;
});
  const bounds = windowBoundaries(o.startTick, o.endTick, o.bpm, changes, WINDOW_SECONDS);
  const scheduleWindow = function (k) {
    const a = bounds[k];
    const b = bounds[k + 1];
    const nowTime = ticksToSecondsWithChanges(a, o.bpm, changes) - ticksToSecondsWithChanges(o.startTic
k, o.bpm, changes);
    if (k === 0) {
      // 시작 위치에 걸쳐 있는 클립은 중간부터 울림
      clipsCovering(audioClips, a).forEach(function (c) { player.schedule(c, a, nowTime, o.bpm, change
s); });
    }
    clipsStartingIn(audioClips, a, b).forEach(function (c) { player.schedule(c, a, nowTime, o.bpm, chang
es); });
    notesStartingIn(noteEvents, a, b).forEach(function (ev) { notePlayer.schedule(ev, a, nowTime, o.bpm,
changes); });
  };
  scheduleWindow(0);

  // 4) 창 경계마다 멈춰서 다음 창을 예약. 경계 시각은 렌더링 샘플 단위로 맞춰짐
  const t0 = ticksToSecondsWithChanges(o.startTick, o.bpm, changes);
  const stops = [];
  for (let k = 1; k < bounds.length - 1; k++) {
    const sec = ticksToSecondsWithChanges(bounds[k], o.bpm, changes) - t0;
    if (sec > 0 && sec < totalSec) stops.push({ k: k, sec: sec });
  }
  let failed = null;
  stops.forEach(function (s) {
    ctx.suspend(s.sec).then(function () {
      if (failed) { ctx.resume(); return; }
      if (cancelled()) { failed = new Error('cancelled'); ctx.resume(); return; }   // 남은 구간은 소리
없이 빠르게 끝내고 결과를 버림
      scheduleWindow(s.k);
      progress(0.1 + 0.9 * (s.sec / totalSec));
      ctx.resume();
    });
  });

  const rendered = await ctx.startRendering();
  if (failed) throw failed;
  progress(1);
  const channels = [];
  for (let c = 0; c < rendered.numberOfChannels; c++) channels.push(rendered.getChannelData(c));
  return { channels: channels, sampleRate: rendered.sampleRate, frames: rendered.length };
}
```

3.3 믹서의 즉시 반영 (`mixer.js`)

```
applyTracks(tracks, immediate) {
  const audible = {};
  audibleTrackIds(tracks).forEach(function (id) { audible[String(id)] = true; });
  const self = this;
  const now = this.context.currentTime;
  // 재생 중에는 뚝 끊기는 소리(클릭)를 막으려고 아주 짧게 이어서 바꾸고, 렌더링 시작 때(immediate)는 바로
그 값으로 둠
  // (뮤트한 트랙이 처음 몇십 ms 동안 새어 나오지 않게 하려고)
  const set = function (param, value) {
    if (immediate) param.setValueAtTime(value, now);
    else param.setTargetAtTime(value, now, 0.01);
  };
  (tracks || []).forEach(function (t) {
    if (t.track_type === 'MASTER') {
      set(self.master.gain, dbToGain(t.volume_db));
      return;
    }
    const s = self.strip(t.track_id);
    const gain = audible[String(t.track_id)] ? dbToGain(t.volume_db) : 0;
    set(s.input.gain, gain);
    if (s.pan) set(s.pan.pan, Math.max(-1, Math.min(1, Number(t.pan) || 0)));
  });
}
```

4. 시간 창 예약

모든 노드를 한꺼번에 만들지 않고, 2초 길이의 창마다 그 창에서 시작하는 소리만 만듭니다.

```
시간(초):   0 ———————— 2 ———————— 4 ———————— 6 — ...
창:        [  창 0    )[  창 1    )[  창 2    )
예약 시점:  렌더링 전    2초에 멈춤   4초에 멈춤      각 지점에서 suspend → 다음 창 예약 → resume
```

경계	처리
창의 경계 tick	`windowBoundaries`가 구합니다. 인접한 창이 같은 경계 값을 쓰므로 소리가 빠지거나 두 번 만들어지지 않습니다.
경계를 올림합니다	경계 tick에서 시작하는 소리가 창이 열리는 시각(`suspend`한 시각)보다 이르면 안 되기 때문입니다. 반올림하면 최대 반 tick(약 0.26ms) 일찍 시작할 수 있습니다.
시작 위치에 걸친 클립	첫 창에서 `clipsCovering`으로 찾아 중간부터 울립니다.
범위 시작 전에 시작한 노트	울리지 않습니다. 실시간 재생이 중간부터 재생할 때 그 노트를 울리지 않는 규칙과 같습니다.
범위 끝을 넘는 클립과 노트	`clipsInRange`가 끝에서 자릅니다. 노트의 릴리스와 이펙트의 꼬리는 꼬리 구간에서 자연스럽게 사라집니다.

5. 최신 문법으로 쓰면

```
// clipsInRange 를 최신 문법으로 쓴 모양
export const clipsInRange = (clips = [], startTick, endTick) =>
```

```
  clips
    .filter((c) => c.start_tick + c.length_tick > startTick && c.start_tick < endTick)
    .map((c) => ({ ...c, length_tick: Math.min(c.start_tick + c.length_tick, endTick) - c.start_tick
}));

// 샘플 디코딩을 병렬로 (실제 코드는 진행률을 알리려고 순서대로 기다립니다)
const entries = await Promise.all(ids.map(async (id) => [id, await ctx.decodeAudioData(await loadSampleB
ytes(id))]));
```

전통 문법 (실제 코드)	최신 문법
`Object.keys(c).forEach`로 복사	객체 전개 `{ ...c }`
`for` 문과 `await`로 하나씩 디코딩	`Promise.all`로 한꺼번에
`o.tailSec || 0`	`o.tailSec ?? 0`

6. 왜 이렇게

선택	이유
재생과 같은 클래스를 그대로 씁니다	내보낸 파일이 화면에서 듣던 소리와 다르면 안 됩니다. 소리를 만드는 코드가 하나이면 둘이 어긋날 수 없습니다. S1-11a에서 모든 클래스가 컨텍스트를 인자로 받게 만든 이유입니다.
창마다 멈춰서 노드를 만듭니다	곡 전체의 노드를 한 번에 만들면 노트가 수천 개일 때 메모리를 많이 쓰고 시작이 느립니다. 또 `startRendering`은 한 번 시작하면 끝까지 가기 때문에, 중간에 진행률을 알리거나 멈출 방법이 `suspend`뿐입니다.
창의 길이는 2초입니다	너무 짧으면 멈추는 횟수가 늘어 느려지고, 너무 길면 진행률이 뚝뚝 끊기고 취소 반응이 느립니다. 12초 곡이면 5번 알립니다.
샘플을 오프라인 컨텍스트로 디코딩합니다	`decodeAudioData`는 디코딩한 소리를 컨텍스트의 샘플레이트로 다시 맞춰 줍니다. 44.1kHz 샘플을 48kHz로 내보내도 음정이 바뀌지 않는다는 것을 시험(440Hz)으로 확인했습니다.
렌더링 시작 때 트랙 값은 즉시 반영합니다 (`immediate`)	실시간 재생은 클릭 소리를 막으려고 10ms에 걸쳐 값을 옮깁니다. 렌더링은 시각 0에서 시작하므로, 이 방식이면 뮤트한 트랙도 처음 수십 ms 동안 새어 나옵니다. 시작할 때는 바로 그 값으로 두어야 합니다. 시험으로 뮤트한 트랙이 처음부터 끝까지 완전한 무음인 것을 확인합니다.
취소하면 남은 구간을 빠르게 끝내고 결과를 버립니다	`OfflineAudioContext`는 도중에 닫을 수 없습니다. 멈춘 채로 두면 `startRendering`이 영원히 끝나지 않으므로, 다음 창의 소리를 더 만들지 않고 `resume`해서 빠르게 마친 뒤 `cancelled` 오류로 알립니다. 취소 반응은 최대 한 창(2초) 늦을 수 있습니다.
샘플 하나를 못 불러와도 나머지를 내보냅니다	한 파일 때문에 곡 전체를 포기하는 것보다, 그 클립만 빠지고 경고를 남기는 편이 낫습니다. 재생(290)도 같은 방식입니다.
꼬리는 계산할 길이에만 더합니다	범위의 끝 뒤에는 새 소리를 예약하지 않고, 이미 울리고 있던 이펙트 꼬리와 노트 릴리스가 사라질 시간만 줍니다.

7. 시험

- `docs/handoff/s1-11b_tests/export_unit.mjs`의 마지막 부분(10건)은 순수 함수를 확인합니다. `clipsInRange`는 범위와 겹치는 클립만 남기고, 끝은 범위 끝에서 자르며, 원본은 바꾸지 않고, 끝과 시작이 정확히 맞닿은 클립은 뺍니다. `windowBoundaries`는 경계가 오름차순이고 중복이 없으며 처음과 끝이 범위와 같고, 짧은 범위는 한 창이며, 템포가 바뀌어도 끝을 넘지 않습니다.
- `docs/handoff/s1-11b_tests/render_offline.mjs` (46건)는 실제 `OfflineAudioContext`로 소리를 그려 숫자로 확인합니다.
    - A: 2초 클립이 88200 프레임, 피크 0.5, 좌우 같음, 440Hz입니다.

    - B: 뮤트한 트랙은 처음부터 완전한 무음, 트랙 볼륨 -6dB와 마스터 -6dB는 피크가 절반, 팬 왼쪽 끝은 오른쪽이 무음, 솔로 트랙(880Hz)만 들립니다.
    - C: 48kHz면 96000 프레임이고 음정은 같습니다.
    - D: 범위 시작 위치에 따라 클립이 중간부터 울리고, 범위 끝에서 소리가 잘리며, 꼬리 구간은 무음이고, 범위 밖의 클립은 소리가 없습니다.
    - E: Synth 노트가 C4(261.6Hz)로 울리고, 여러 시간 창(0초, 2초, 4초)의 노트가 모두 울리며, 시작 전에 시작한 노트는 울리지 않고, 끝을 넘는 노트는 잘립니다.
    - F: 12초 클립에서 모든 0.1초 구간에 소리가 있습니다. 창 경계에서 끊기지 않습니다.
    - G: 진행률은 5번 이상 줄지 않고 알려 주며 마지막이 1입니다. 취소하면 `cancelled` 오류입니다.
    - H: 딜레이와 리버브의 꼬리가 범위 끝 뒤에 남고, 이펙트가 없으면 꼬리 구간은 무음입니다.
    - I: 60BPM이면 1920tick이 2초이고, 템포 변경도 반영합니다.
    - J: WAV로 내보낸 뒤 브라우저가 다시 읽은 값이 같습니다 (297에서 설명).
- 시험 중 발견한 일: 템포 변경 시험이 한 번 실패했습니다. 120BPM에서 1초 뒤 240BPM이 되는 곡의 길이를 1.25초로 계산했는데, 1920tick은 240BPM에서 0.5초이므로 1.5초가 맞습니다. 렌더링 결과(1.5초)가 옳았고 시험의 계산이 틀렸습니다.